

CMPT-276 Assignment-4 Report

Ajay Partap Singh Sekhon, 301600629

Manya Asri, 301579204

Introduction:

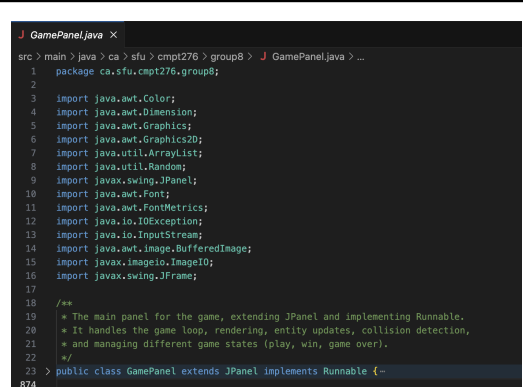
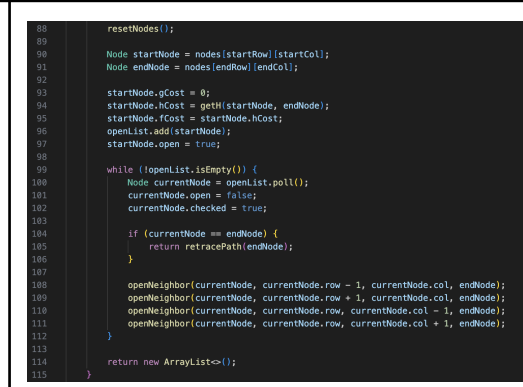
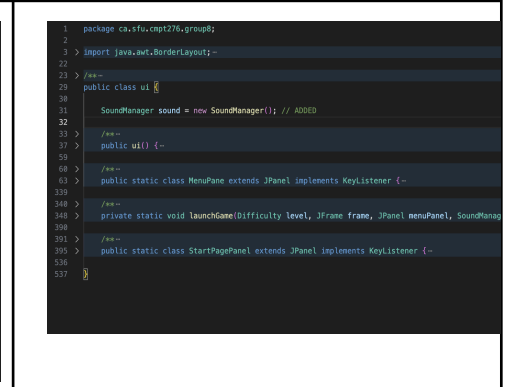
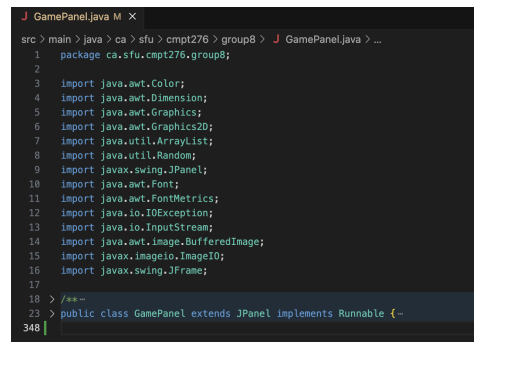
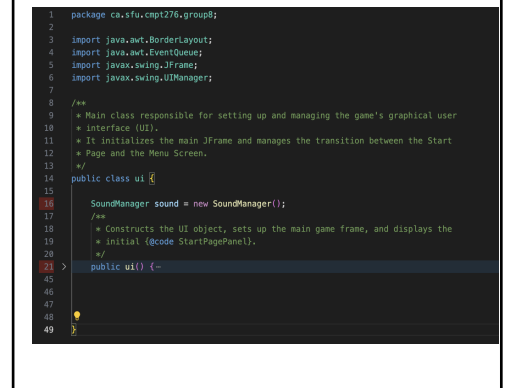
For this code review, we went through our game project carefully to spot areas where the code could be improved. We looked at several classes and found issues like God Classes, repeated code, low cohesion, long methods, and overuse of primitive values. We refactored these parts to make the code easier to read, maintain, and extend. After each change, we ran tests to make sure everything still worked correctly.

The Bad Smells and their fixes:

File Name:	Bad Smell:	Solution:	Commit Reference(s):
UI.java	Class too large, Low Cohesion	UI.java had 2 smaller classes inside, so split UI.java into 4 classes UI.java itself, MenuPane.java, launchGame.java and StartPagePanel.java.	417aa056ec5eef320f9e9a626152e516a3470b9a;7b2562d1b22a38ce35b7cb40dcc5f70edc973b23
TileManager.java	Code duplication	The getTileImage() method had repetitive code, so a helper function named createTile() was made which is also more future proof in case we want to add more tiles.	9fcb989fb9b8a50317255db307623a421c5d36c5
EntitySpawner.java	God Class (GamePanel.java) , Low Cohesion , Feature Envy, Tight Coupling, Duplicated Code, Shotgun Surgery	Refactored spawning logic into a dedicated EntitySpawner class to declutter and focus GamePanel. Centralized randomization and entity-specific spawn methods minimize coupling and duplication, making the design far easier to maintain.	6f7103b7e17968aefcf70f7448bc2ba264c1a627
PathFinder.java	Unjustified use of primitives (Before: Used separate int parameters for coordinates After: Used Positionobjects to encapsulate coordinates in findPath), Long parameter, Duplicated boundary checking, Duplicated code	These refactorings boosted clarity and maintainability. We introduced Position objects to clean up parameters and improve data safety. Helper methods, like isValidPosition() and openAllNeighbors(), now centralize repeated logic. The class adheres better to the Single Responsibility Principle (SRP).	4c9db094dc4ca3ef724150d25fb19b14e818fc83
Player.java	Code duplication, magic number, and inconsistent field access	Added extract helper methods to reduce code duplication and make it more future proof. Added a constant to fix the magic number situation and instead of using direct field access in the draw() function we use the new GameDimensions class instead.	2e3738b3c310ea3a0c8eb2cd a779a48122fe664f
DrawScreen.java	God class , Low cohesion , long method , large class , shotgun surgery , feature envy.	We moved all drawing and UI rendering code into a new class. This keeps GamePanel focused on gameplay logic, improving cohesion and reducing the God Class problem. UI updates now happen in one place, fixing Shotgun Surgery. It also removes Feature Envy because drawing behavior now belongs to the class actually responsible for rendering.	80a0fa3c8a8a68225ac21a d8f7c585d80b2cc159

GameDimensions.java	2 classes used the exact same data clumps (Player.java and GamePanel.java).	Made another class (GameDimensions.java), to encapsulate all dimension data in one place. GamePanel owns the GameDimensions object and shares it via getDimensions(). Player now gets all the dimensions from GamePanel when needed.	0e58ac8274ae005fb19291a5a18e76340981b25e
Difficulty.java	God Class (GamePanel doing too much), Duplicated Code,Poor Cohesion, Hard to Scale, Feature Envy, removed unnecessary if/else from GamePanel	Everything about difficulty (zombie count, speed, spawns) is now in the dedicated Difficulty class, pulling it out of GamePanel. This centralized the data, eliminated repeated checks and duplicated code, and resolved the Feature Envy smell in GamePanel.	da19365d5d63f8b3fb9bfdada3f07c03c30fcb2c

Images of some major refactorings:

		
Before: GamePanel: 874 lines	Before: Used separate int parameters for coordinates	Before: ui: 538 lines
		
After:GamePanel: 348 lines	After: Used Positionobjects to encapsulate coordinates	After :ui: 45 lines

Conclusion: The refactoring also tackled issues like magic numbers, tight coupling, shotgun surgery, and inconsistent field access. We added helper methods, constants, and dedicated classes to reduce duplication and organize responsibilities better. These changes make the code easier to understand, extend, and debug. Overall, the project is now cleaner and more maintainable for future work.